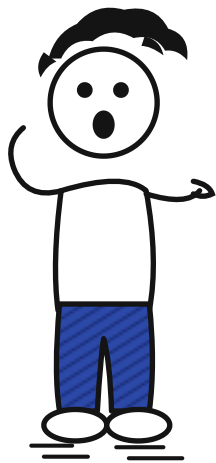
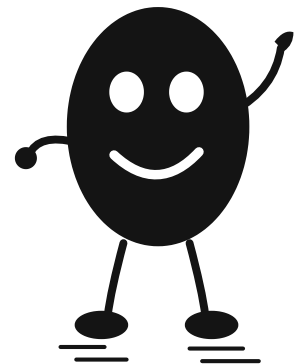


I get what `/loop` does now.

But Boris uses it to have Claude maintain his **whole** app – how does that work?



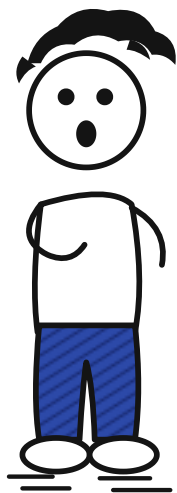
That's called
Loop Engineering.
Come on, I'll
show you.



Hold on – I already know
`/loop`. So what is
Loop Engineering then?

`/loop` is just the building block: a
loop that repeats a task.

Loop Engineering is the craft around it
– making sure the loop can run
unattended for days without causing
harm.



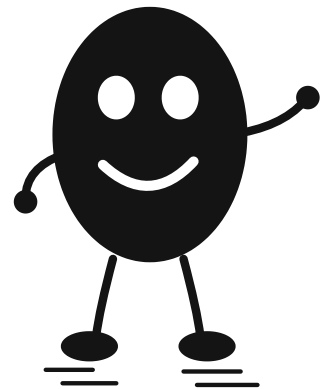
`/loop`

the building block



Auto-pacing +
verification + tuning +
isolation

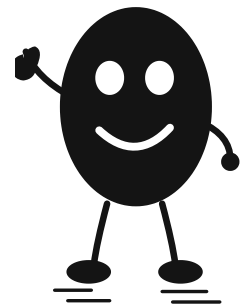
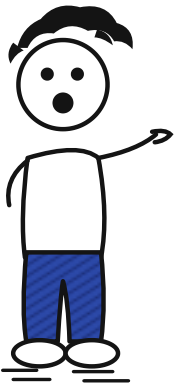
the craft around it



Give me an example of what that looks like in practice.

Boris Cherny runs several routines daily over his own apps, in a Slack channel.

A crash fuzzer taps around the app and fixes crashes, a dup unifier finds duplicate code, a dead-code remover cleans up – each routine with its own job.



Crash fuzzer

Dup unifier

Dead-code remover

Abstraction police



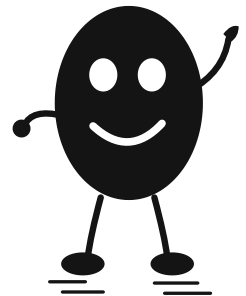
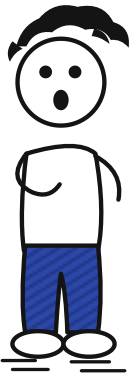
388 pull requests in a few weeks

180 of them merged after code review + human review

Is `/loop` the only way
to run something
like that?

No – there are four kinds of loop. They
differ in **how much you hand off**.

First just the check, then the stop
condition, then the trigger, finally the
task itself. Each step means
less watching over it.

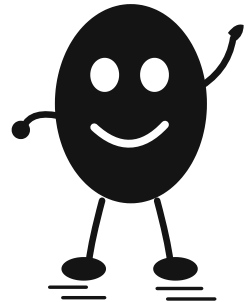
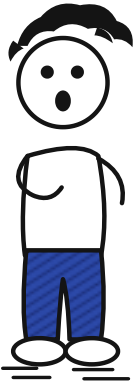


this booklet works through the four steps in order

Do I have to tell the loop every time how often it should run?

Not necessarily. Without a time given, Claude picks the interval itself.

Short while things are actually happening – longer during quiet stretches. That's exactly what makes running for days practical: nobody has to keep adjusting an interval.



busy
short interval

quiet stretch
long interval

blocked
fallback wake-up

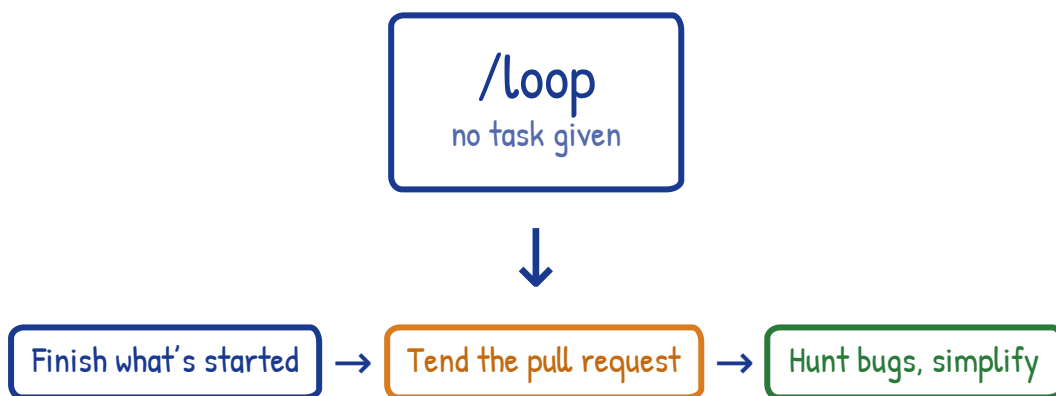
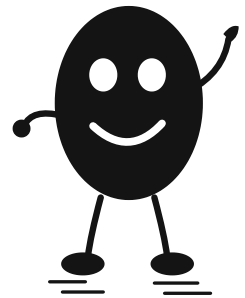
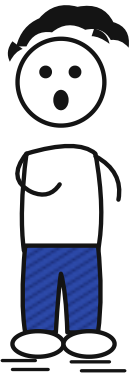


Claude decides for itself when the next run is worth it

And what if I type
`/loop` with no task
at all?

Then a built-in **maintenance task** kicks in: finish what was started, tend the open pull request, and when nothing is pending, hunt bugs and simplify.

Put your own rules in a `loop.md` file – then a bare `/loop` does exactly that. Boris' maintenance idea, built in.



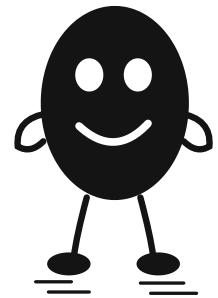
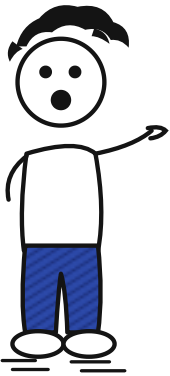
🔄 at its own pace

your own version: `.claude/loop.md` in the project

This loop that just keeps planning, doing, checking – does that have a name too?

Yes – that's called a **Ralph-loop**: plan, execute, check, from the top.

Robust, but crude – without a brake it keeps burning tokens even after it's long since done. That's exactly what auto-pacing is for, and the verification that comes next.



↻ from the top, until done

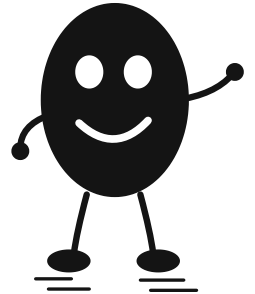
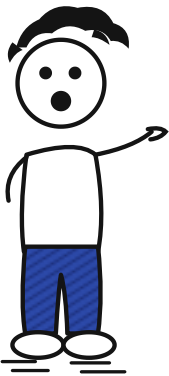
Ralph loop

robust, but expensive in tokens without a brake

And that Ralph loop – do I have to build it myself?

No, it's built in: `/goal`. You don't say *how often* it runs, you say **how to tell that it's done**.

After each round a second, small model reads the transcript and returns a verdict: not yet, met, or impossible. Only the first one keeps it going – that's the brake the Ralph loop was missing.



`/goal all tests pass`
or stop after 20 rounds



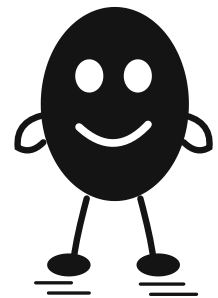
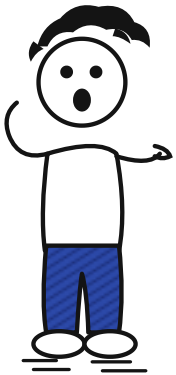
↻ not met yet

the evaluator only reads what Claude itself has shown

But if nobody's watching
- how am I supposed to
trust the loop?

That's exactly where Loop Engineering
stands or falls: the loop has to be able
to verify its own work **itself**,
end-to-end.

Run the tests, have a second model
review it - automated code review and
security review - and only then offer
the change as a pull request.



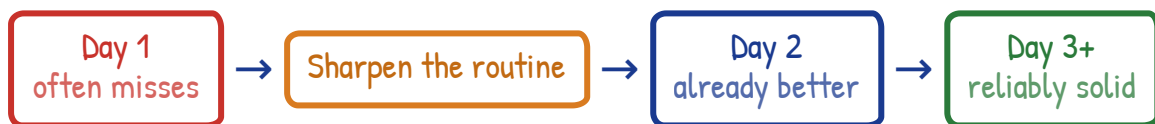
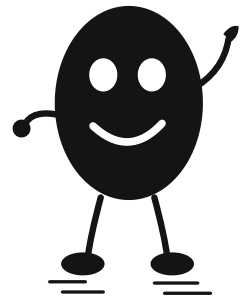
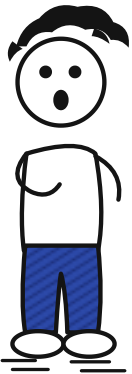
if a check fails, there is no proposal - no human has to look at it beforehand

no tests? Then two independent routes
how Claude backed up a nine-loop physics calculation in 2026

And what if the loop makes a mess of it at first?

Then you adjust the routine – or just ask Claude to improve it itself.

Sometimes one day is enough, sometimes it takes several attempts before the routine sits reliably. That's expected, not a failure.

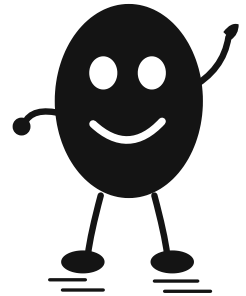
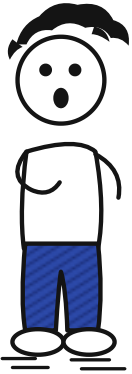


Claude gets feedback and tunes its own routine

Is that tuning from before systematic over time, or is it just guesswork?

The systematic version is called **hill climbing**: you measure quality with a fixed checklist – an eval – and adjust the routine on purpose.

No guesswork: every change gets tested against the eval, only what actually scores better stays in.



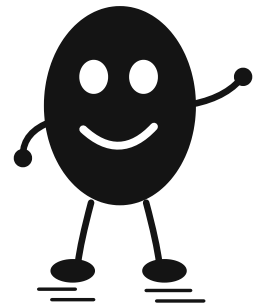
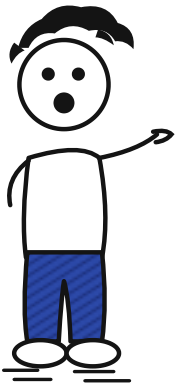
↗ step by step, uphill

keep only what the eval confirms – that's hill climbing

Don't several loops in the same folder get in each other's way?

No – each routine works in its own copy of the project, a **worktree**.

That way parallel runs don't interfere with each other, and a failed attempt doesn't drag the main branch down with it.



Main project



Worktree A

Worktree B

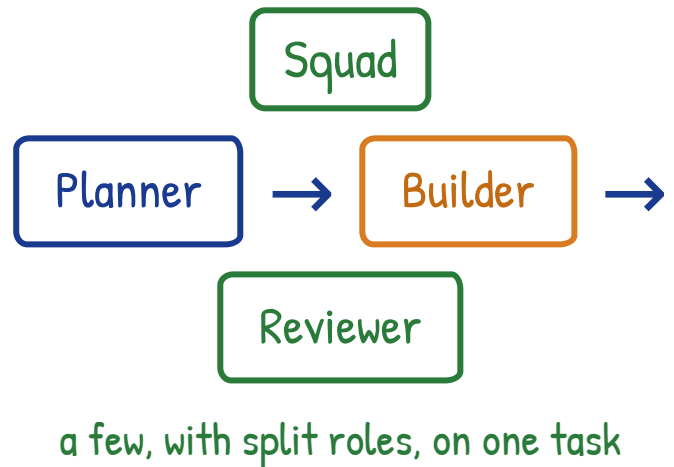
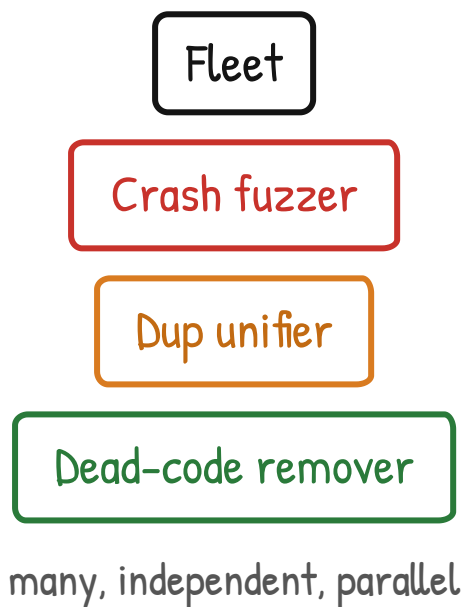
Worktree C

each loop on its own, nothing overwrites anything else

The worktrees from before – do they each run as their own, separate Claude?

Mostly yes: each routine its own Claude – many of them at once is called a **fleet**.

But sometimes several Claudes work **together** on a single task, with roles split up – that's a **squad**: one plans, one builds, one reviews.



This all runs on my machine – so what if I close the laptop?

Then it's over. `/loop` needs a running Claude Code session and ends after seven days at the latest.

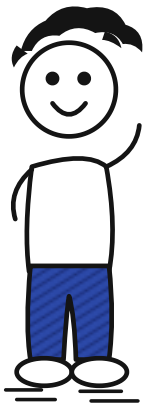
In between sit **Desktop tasks**: local, with your files, no open session needed – as long as the app is running. And `/schedule` moves the routine to the cloud – it runs with the laptop closed, but without local files and without asking.



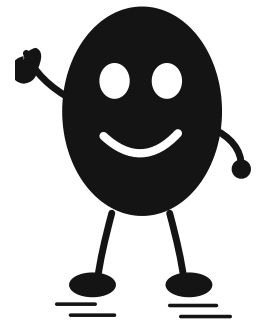
And the fourth step -
when nobody types
anything anymore?

Then the work starts by itself: at a set
time, on a GitHub event, or when
another program knocks at the
webhook address.

`/schedule` supplies the trigger,
`/goal` the target, verification the
brake, auto mode the freedom from
prompts. Each task ends at its goal -
the routine runs until you switch it off.



`/schedule + /goal +
verification + auto mode`

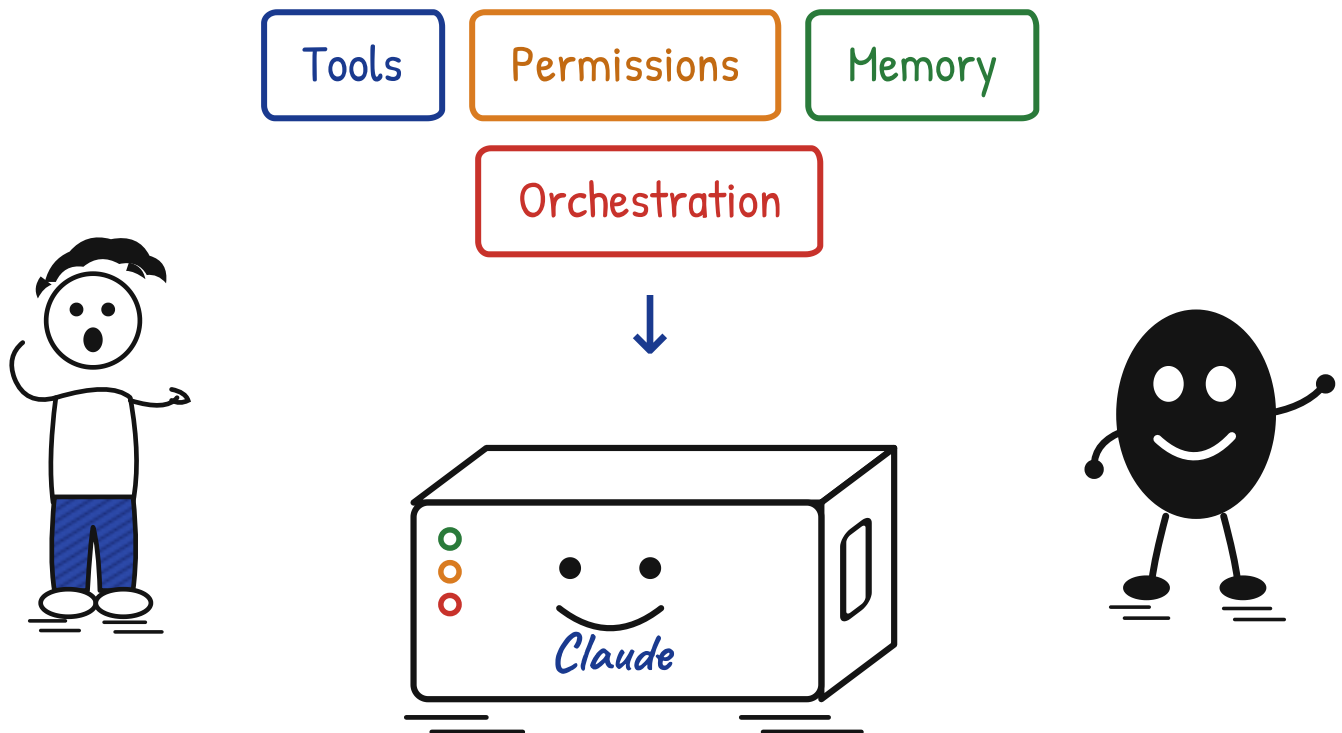


nobody types anything anymore

Auto-pacing, verification, tuning, isolation – does this whole scaffolding have a name too?

Yes – the sum of tools, permissions, memory, and orchestration around the model is called a **harness**.

Loop Engineering is exactly that: building a harness Claude can work in, unattended, for days.

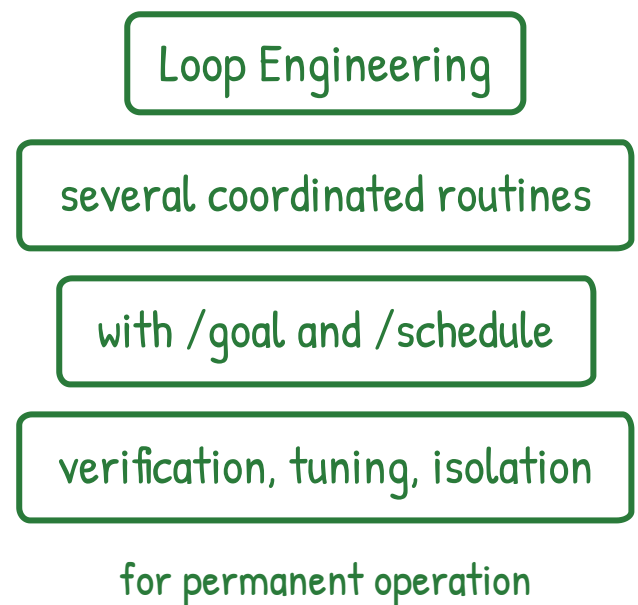
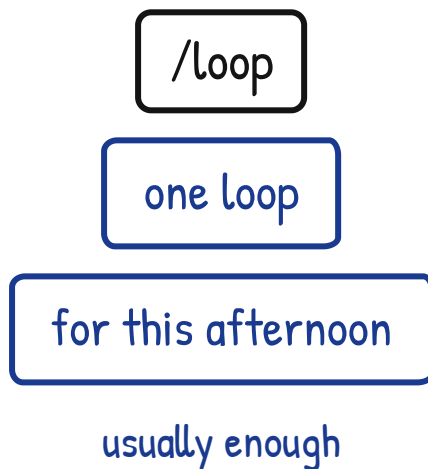


all of it together: the **harness**

So when is all this craft actually worth it – isn't /loop enough most of the time?

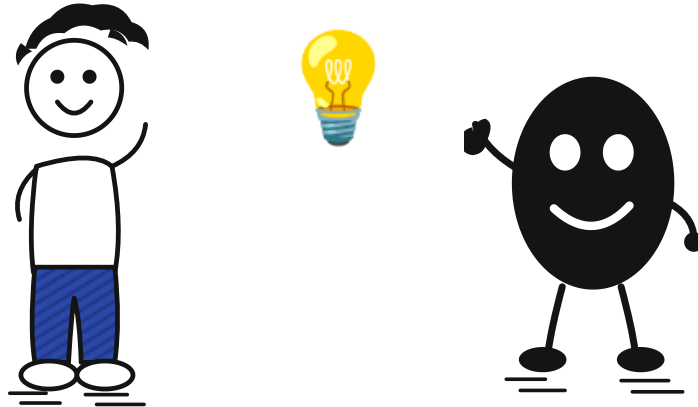
For a single afternoon, /loop really is completely enough.

Loop Engineering pays off once several such routines are meant to run alongside each other permanently – then individual loops turn into a small workshop that maintains itself.



Now Boris' Slack channel finally makes sense.

That's exactly the point.



Four loops - check, condition, trigger, task
 /loop - the building block, repeats a task
 /loop with no task - built-in maintenance, your own via loop.md
 /goal - the condition instead of the interval
 /schedule - the same routine, in the cloud
 Ralph loop - plan, execute, check, from the top
 Auto-pacing - Claude picks the interval itself
 Verification - tests, review, only then a proposal
 Hill climbing - tuning with an eval instead of guesswork
 Worktree isolation - each loop on its own
 Squad / fleet - split roles, or many in parallel
 Harness - tools, permissions, memory, orchestration
 Proactive - trigger, target, brake - entirely without you
 Loop Engineering - all of it as a workshop in permanent operation